

Migaki: Toward an Execution Optimizer for Agentic Systems

Draft Vision Paper v0.3

Aleksandr Lopashev
Migaki Project
`migaki-dev/migaki`

June 2026

Abstract

Foundation models have become increasingly capable. Agent frameworks have become increasingly sophisticated. Execution has become increasingly fragmented. Modern AI systems already contain many forms of execution infrastructure: agent SDKs, graph runtimes, provider gateways, tool protocols, durable workflow engines, caching layers, observability platforms, and evaluation systems. The problem is no longer that execution infrastructure does not exist. The problem is that execution infrastructure does not compose.

Routing lives in gateways. State lives in frameworks. Retries live in workflow engines. Context management lives in provider APIs. Verification lives in evaluation platforms. Tool interoperability lives in protocol layers. Tracing lives in observability systems. Each layer optimizes locally. Few systems represent the full model/tool/retrieval/context workflow as a portable execution object that can be transformed, evaluated, lowered, replayed, and audited.

This paper proposes MIGAKI, an execution optimizer for agentic systems. MIGAKI separates what an AI system intends to compute from how that computation is realized across models, providers, tools, context, memory, caches, validators, and runtimes. It introduces MIR, a provider-neutral intermediate representation for model/tool/retrieval/context workflows. Applications and frameworks compile into MIR. Optimization passes transform MIR under explicit cost, latency, quality, policy, and governance constraints. Runtime backends lower optimized plans into provider-specific APIs and execution environments. Every transformation emits evidence traces so execution optimization remains measurable rather than opaque.

1 Introduction

The current AI application stack is crowded. A production agentic system may include an application framework, an agent orchestration library, a model gateway, one or more provider APIs, tool protocols, durable workflow infrastructure, observability, and evaluation. OpenAI's Agents SDK, for example, exposes agents, tools, handoffs, guardrails, sessions, human-in-the-loop mechanisms, and tracing in Python and JavaScript/TypeScript [1–3]. LangGraph emphasizes durable execution, streaming, persistence, and human-in-the-loop workflows [8]. Cloudflare AI Gateway provides analytics, logging, caching, rate limiting, retries, and model fallback [12]. LiteLLM exposes a provider gateway abstraction and model routing strategies across many providers [11]. The Model Context Protocol (MCP) standardizes tool exposure through named tools with schemas and metadata [16]. OpenTelemetry has moved GenAI telemetry toward specialized semantic conventions for spans, metrics, and events [17].

This is progress. It is also fragmentation. Execution decisions are scattered across application code, framework internals, provider-specific APIs, gateway settings, workflow retries, evaluation jobs,

and tracing backends. The same concerns recur: model choice, provider choice, context assembly, duplicate elimination, prompt-cache layout, retrieval ordering, side-effect control, validation, retry policy, human approval, trace capture, cost accounting, and data governance.

MIGAKI begins from a narrow claim: agentic systems need an execution layer that represents model calls, tool calls, retrieval, context artifacts, validators, side effects, and constraints as an optimizable execution graph. The goal is not to build another agent framework. The goal is to make AI execution portable, measurable, and optimizable.

2 The Missing Middle

Agent frameworks decide what should happen next. Gateways route model requests. Workflow engines preserve durable state. Provider APIs expose model-specific capabilities. Tool protocols expose external systems. Observability platforms record behavior. Evaluation systems measure outcomes.

Each layer owns part of execution. Few layers can optimize across the whole execution graph. A single agentic workflow may require decisions across all of the following:

- model selection and provider routing;
- retry, fallback, and timeout policy;
- retrieval ordering and context assembly;
- duplicate context elimination and prompt-cache layout;
- tool-call parallelization and side-effect control;
- validation, regression testing, and human approval;
- cost accounting, latency budgeting, and audit replay;
- provider allowlists, data-retention rules, and trace redaction.

These decisions interact. A cache decision may depend on context layout. A routing decision may depend on privacy class. A retry policy may depend on idempotency. A validation policy may change whether lossy compression is acceptable. Local optimization is therefore insufficient.

Claim 1 (Execution graph as optimization boundary). *The unit of optimization in agentic systems should not be a single prompt or model call. It should be the full model/tool/retrieval/context execution graph.*

3 Position Within the Ecosystem

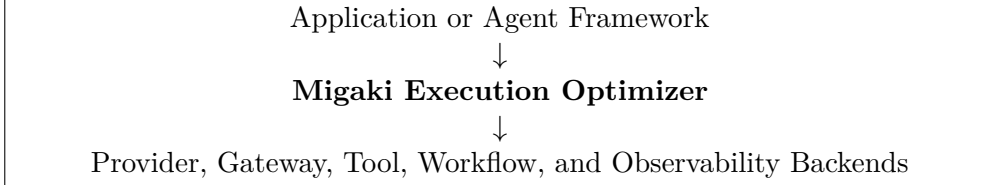
MIGAKI is not an agent framework. Agent frameworks model planning, orchestration, tool loops, memory, user interaction, and application-specific control flow. OpenAI Agents SDK, LangGraph, Google ADK, and Microsoft Agent Framework all target parts of this space [2, 8–10].

MIGAKI is not an AI gateway. Gateways provide provider abstraction, load balancing, fallbacks, caching, rate limits, budgets, and monitoring. LiteLLM, Cloudflare AI Gateway, and Vercel AI Gateway already cover substantial parts of this surface [11–14].

MIGAKI is not a durable workflow engine. Durable runtimes such as Temporal preserve progress across failures and support long-running workflows, including human-in-the-loop workloads [15]. MIGAKI may integrate with these systems, but does not replace their persistence semantics.

MIGAKI is not an observability platform. Platforms such as Langfuse provide tracing, debugging, prompt management, latency and cost tracking, datasets, and evaluation workflows [18]. MIGAKI should export evidence to such systems rather than reinvent them.

MIGAKI occupies the missing middle:



It consumes a logical execution graph, applies optimization passes, lowers the result into concrete runtimes, and emits evidence.

4 From Compiler Analogy to Probabilistic Query Optimization

Computing repeatedly separates intent from execution. Source code is separated from machine code by compilers. SQL queries are separated from execution plans by query optimizers. Applications are separated from CPU scheduling by operating systems. Distributed APIs are separated from transport mechanics by RPC runtimes.

The compiler analogy is useful but limited. Traditional compilers typically preserve exact program semantics. AI execution is stochastic: changing context order, compressing history, switching models, altering retrieval, or delaying reasoning can change output behavior.

MIGAKI is therefore closer to a query optimizer for probabilistic model/tool graphs. A query optimizer receives a logical query, estimates alternatives, and produces an execution plan. MIGAKI receives a logical AI workflow, estimates alternatives, and produces an execution plan under cost, latency, quality, policy, and provider-capability constraints.

The goal is not exact semantic preservation. The goal is validated behavioral equivalence under declared constraints.

5 Formal Model

Let an agentic workflow be represented as a directed acyclic or resumable graph

$$G = (V, E), \tag{1}$$

where each node $v \in V$ is typed by

$$\tau(v) \in \{\text{MODEL CALL}, \text{TOOL CALL}, \text{RETRIEVAL}, \text{TRANSFORM}, \text{VALIDATOR}, \text{HUMAN APPROVAL}, \text{CACHE READ}, \text{CACHE WRITE}\} \tag{2}$$

The graph is logical: it describes what computation should occur, not how it should be realized.

An execution plan π assigns execution decisions to G :

$$\pi = (m, r, s, c, a, h), \tag{3}$$

where m assigns models/providers to model-call nodes, r assigns retry and fallback policies, s assigns a schedule, c assigns cache behavior, a assigns context actions such as keep/drop/deduplicate/compress, and h assigns human-approval and validation gates.

Let $\Pi(G, \mathcal{K})$ denote the set of feasible plans for G under system constraints $\mathcal{K}$. Let π_0 denote the baseline plan authored by the application or framework. A MIGAKI optimizer searches for

$$\pi^* = \arg \min_{\pi \in \Pi(G, \mathcal{K})} \lambda_C \hat{C}(\pi) + \lambda_L \hat{L}_{p95}(\pi) + \lambda_R \hat{R}(\pi) + \lambda_P P(\pi), \quad (4)$$

subject to

$$Q(\pi; \mathcal{T}, \mathcal{V}) \geq Q(\pi_0; \mathcal{T}, \mathcal{V}) - \epsilon_Q, \quad (5)$$

$$\text{Policy}(\pi) \models \mathcal{K}, \quad (6)$$

$$\hat{C}(\pi) \leq B, \quad (7)$$

$$\hat{L}_{p95}(\pi) \leq S. \quad (8)$$

Here $\hat{C}$ is estimated or measured monetary cost, $\hat{L}_{p95}$ is tail latency, $\hat{R}$ estimates quality or operational risk, and P captures policy penalty terms. The quality constraint in Equation 5 is deliberately validator-based, not confidence-based.

This framing avoids the vague claim of “same answer, lower cost.” A defensible claim is instead:

Equivalent task outcome under declared validators and allowed regression thresholds,
with lower cost, lower latency, or improved reliability.

6 mIR: Migaki Intermediate Representation

MIGAKI introduces MIR, a provider-neutral intermediate representation for AI execution. MIR is not a prompt format. It is not a JSON wrapper around chat completions. It is a logical representation of agentic computation.

At minimum, MIR should represent:

- model calls, tool calls, and retrieval calls;
- context artifacts and memory reads/writes;
- cache reads/writes and provider-specific cache annotations;
- transforms, validators, branches, joins, retries, and fallbacks;
- human approval gates and side-effect annotations;
- observability hooks and execution constraints.

The key design principle is that context is a first-class object. Most execution waste in AI systems is caused not only by final model calls, but by repeated instructions, duplicated retrieved chunks, irrelevant history, oversized memory, unstructured summaries, and failure to exploit provider-specific cache behavior.

```

type MIRPlan = {
  id: string
  version: "migaki.mir.v0"
  nodes: MIRNode[]
  edges: MIREdge[]
  context: MIRContextBlock[]
  constraints: MIRConstraints
}

type MIRContextBlock = {
  id: string
  role: "system_instruction" | "developer_instruction" |
    "user_input" | "retrieved_document" | "memory" |
    "tool_result" | "example" | "scratchpad" |
    "validator_output"
  contentRef: string
  provenance: Provenance
  tokenEstimate?: number
  mutability: "fixed" | "droppable" | "deduplicable" |
    "compressible" | "summarizable"
  cachePolicy?: CachePolicy
  privacyClass?: PrivacyClass
  retentionPolicy?: RetentionPolicy
}

```

7 Context Selection and Deduplication

Let a workflow expose candidate context blocks

$$X = \{x_1, x_2, \dots, x_n\}. \quad (9)$$

Each block has token length t_i , utility estimate u_i , role ρ_i , mutability class μ_i , and privacy class κ_i . A context-selection pass chooses binary variables $z_i \in \{0, 1\}$:

$$\max_{z \in \{0,1\}^n} \sum_{i=1}^n z_i u_i - \gamma \sum_{i < j} z_i z_j d_{ij} \quad (10)$$

subject to

$$\sum_{i=1}^n z_i t_i \leq T, \quad (11)$$

$$z_i = 1 \quad \forall i \in R, \quad (12)$$

$$z_i = 0 \implies \mu_i \in \{\text{DROPPABLE}, \text{COMPRESSIBLE}, \text{SUMMARIZABLE}\}. \quad (13)$$

Here d_{ij} penalizes redundant context and R is the set of required blocks. Exact duplicate elimination is the safest first pass: if $H(x_i) = H(x_j)$ for a canonical content hash H , and neither block carries unique provenance required by policy, then one copy can be removed while preserving a provenance join.

Lossy compression is different. A compression transform g maps x_i to $g(x_i)$ with $t(g(x_i)) < t_i$. It should be admitted only when a validator accepts it:

$$\text{Accept}(g(x_i)) = 1 \iff \mathcal{V}_i(g(x_i), x_i) = 1 \wedge \text{Policy}(g(x_i)) \models \mathcal{K}. \quad (14)$$

This principle is intentionally conservative: never perform lossy optimization without a declared acceptance test.

8 Cache Planning

Provider APIs increasingly expose prompt caching behavior. OpenAI documents automatic prompt caching that can reduce latency and input token costs for repeated prompt prefixes [5]. Anthropic documents prompt caching with automatic caching, explicit cache breakpoints, TTL behavior, and cache-control semantics [6, 7].

For a model-call node v , let its input be divided into cached-read tokens h_v , cache-write tokens w_v , fresh input tokens f_v , and output tokens o_v . A simple provider-specific cost estimate is

$$\hat{C}_v = c_{p(v)}^{\text{read}} h_v + c_{p(v)}^{\text{write}} w_v + c_{p(v)}^{\text{input}} f_v + c_{p(v)}^{\text{output}} o_v, \quad (15)$$

where $p(v)$ is the provider/backend chosen for node v . A cache-planning pass minimizes expected cost and latency by arranging stable context in prefixes, preserving repeated system/developer instructions, and placing provider-specific cache boundaries when supported:

$$\min_c \sum_{v \in V_{\text{model}}} \mathbb{E}[\hat{C}_v(c)] \quad \text{subject to} \quad c \in \mathcal{C}_{p(v)}. \quad (16)$$

Here $\mathcal{C}_{p(v)}$ represents the cache capabilities of the selected provider, such as automatic prefix caching or explicit breakpoints.

9 Routing Under Constraints

Routing is a natural optimization pass, but it is easy to overclaim. Research on LLM routing studies the cost-quality tradeoff between stronger and weaker models. RouteLLM learns routers from preference data and reports cost reductions under benchmarked conditions [19]. LLMRouter-Bench provides a large benchmark for routing across datasets and models and warns that many approaches perform similarly under unified evaluation, with some commercial routers failing to reliably outperform simple baselines [20].

For a model-call node v , let $\mathcal{M}_v$ be the feasible model/provider set after policy and capability filtering. A routing pass may choose

$$m^*(v) = \arg \min_{m \in \mathcal{M}_v} \hat{c}(v, m) \quad (17)$$

subject to

$$\hat{q}(v, m) \geq \theta_v, \quad (18)$$

$$\hat{\ell}(v, m) \leq s_v, \quad (19)$$

$$\text{allowed}(m, \kappa_v, \mathcal{K}) = 1. \quad (20)$$

Here $\hat{c}$ is estimated cost, $\hat{q}$ is estimated task-specific quality, $\hat{\ell}$ is estimated latency, θ_v is a quality threshold, s_v is a latency service level, and κ_v is the privacy class of the data entering node v .

A MIGAKI router is not credible because it routes. It is credible only if it reports measured deltas against baselines and validates that quality constraints still hold.

Pass family	Examples
Context	exact duplicate elimination, dead context removal, tool definition pruning, stable prefix construction, memory compaction planning
Cache	prompt-cache layout, breakpoint placement, cache hit estimation, cache read/write reporting
Routing	model selection, provider selection, fallback planning, rate-limit-aware routing, cost-aware routing, latency-aware routing
Parallelism	independent retrieval parallelization, parallel tool execution, speculative branch execution, validator parallelization
Verification	deterministic invariants, schema validation, source-grounding checks, task-specific validators, model-based judges
Reliability	retry planning, fallback planning, partial replay, idempotency checks, human approval insertion
Governance	provider allow/deny enforcement, data-retention enforcement, privacy-class routing, trace redaction

Table 1: Representative MIGAKI optimization pass families.

10 Provider Neutral, Capability Aware

MIGAKI is provider-neutral, but it should not be provider-naive. Different providers expose different capabilities: prompt caching, explicit cache breakpoints, tool calling, remote MCP support, code execution, file search, web search, structured outputs, reasoning controls, batch APIs, rate-limit semantics, data-retention modes, region controls, and tracing integrations.

A lowest-common-denominator abstraction would erase useful execution capabilities. Instead, MIR should remain portable while lowering remains capability-aware:

```
type ProviderCapabilities = {
  supportsPromptCaching: boolean
  supportsExplicitCacheBreakpoints: boolean
  supportsToolCalling: boolean
  supportsRemoteMCP: boolean
  supportsStructuredOutputs: boolean
  supportsBatching: boolean
  supportsReasoningControls: boolean
  supportsZeroDataRetention?: boolean
  maxContextTokens?: number
  cacheTtlOptions?: string[]
}
```

Provider backends lower optimized MIR into concrete execution plans. In the near term, providers do not execute MIR directly. MIGAKI executes through adapters targeting provider APIs, gateways, workflow runtimes, or local inference backends.

11 Optimization Passes

A MIGAKI execution plan is transformed by passes over MIR. Examples include:

Each pass should emit a plan diff and evidence. A minimal pass interface is:

```
interface MigakiPass {
```

```

name: string
version: string
apply(plan: MIRPlan, context: PassContext): Promise<PassResult>
}

type PassResult = {
  plan: MIRPlan
  diff: MIRPlanDiff
  evidence: Evidence[]
  warnings: Warning[]
}

```

12 Verification and Evidence

Execution optimization must remain measurable. Every optimization should produce evidence. MIGAKI should emit an Execution Evidence Bundle containing the original plan, optimized plan, plan diff, enabled passes, disabled passes, context changes, token estimates, cache decisions, routing decisions, retry and fallback decisions, validator results, cost and latency deltas, trace identifiers, replay handles, policy decisions, human approvals, and redactions.

Given a baseline plan π_0 and optimized plan π , define an evidence tuple

$$\mathcal{E}(\pi_0, \pi) = (\Delta C, \Delta L, \Delta Q, \mathcal{D}_{\text{plan}}, \mathcal{R}_{\text{validators}}, \mathcal{R}_{\text{policy}}, \mathcal{T}_{\text{trace}}), \quad (21)$$

where

$$\Delta C = \hat{C}(\pi_0) - \hat{C}(\pi), \quad \Delta L = \hat{L}(\pi_0) - \hat{L}(\pi), \quad \Delta Q = Q(\pi) - Q(\pi_0). \quad (22)$$

The optimized plan is acceptable only if

$$\text{Accept}(\pi \mid \pi_0) = 1 \quad (23)$$

where

$$\text{Accept}(\pi \mid \pi_0) = 1 \iff \mathcal{R}_{\text{validators}} = 1 \wedge \Delta Q \geq -\epsilon_Q \wedge \mathcal{R}_{\text{policy}} = 1. \quad (24)$$

Evidence is central. Without evidence, optimization becomes invisible prompt mutation. Invisible prompt mutation is not infrastructure.

13 Examples

13.1 RAG Workflow

A baseline enterprise support workflow may retrieve 20 document chunks, send all chunks to a frontier model, generate an answer, and return it. A MIGAKI-optimized plan may deduplicate overlapping chunks, drop low-relevance chunks, preserve stable policy instructions as a cacheable prefix, route chunk ranking to a cheaper model, use a stronger model for synthesis, validate the answer against retrieved sources, retry only the synthesis step if validation fails, and emit an evidence bundle.

The agent framework still decided the strategy:

Retrieve $\rightarrow$ Answer $\rightarrow$ Validate.

MIGAKI optimized the execution:

Deduplicate $\rightarrow$ Cache $\rightarrow$ Route $\rightarrow$ Parallelize $\rightarrow$ Validate $\rightarrow$ Report.

13.2 Code Review Workflow

A code-review agent may load repository context, changed files, style guides, and prior conversation, then ask a frontier model for review. A MIGAKI plan can mark style guides as fixed and cacheable, mark changed files as non-droppable, remove unrelated history, run deterministic static checks, route lint explanations to a cheaper model, use a stronger model for architectural review, validate final comments against changed lines, and export evidence.

This makes execution policy explicit and measurable: baseline tokens, optimized tokens, baseline cost, optimized cost, latency, validator pass rate, false-positive rate, and comment acceptance rate.

14 Initial Scope

The first implementation should be intentionally small. The objective is not completeness. The objective is validating the abstraction.

MIGAKI v0 should include:

- TypeScript runtime and fetch-compatible provider wrapper;
- MIR schema v0 and provider capability registry;
- adapters for OpenAI-style, Anthropic-style, and LiteLLM-compatible APIs;
- context block ledger, token estimation, and cost estimation;
- exact duplicate context elimination and stable prefix detection;
- prompt-cache layout reporting;
- simple retry, fallback, and static routing policy;
- execution evidence bundle and replayable trace artifact;
- CLI report for baseline-versus-optimized execution.

It should not lead with general semantic compression, general semantic caching, autonomous global routing, cross-provider equivalence, or claims of identical answers. These can be experimental passes later.

A credible v0 demo is a multi-step RAG or code-review workflow in which MIGAKI reduces context duplication, improves cache layout, routes cheap classification/ranking calls to cheaper models, keeps synthesis on stronger models, validates structured output, retries only failed branches, and emits an evidence report showing cost, latency, and validator deltas.

15 Why Now

Several trends make execution optimization timely.

First, provider fragmentation is increasing. Organizations use multiple frontier providers, local models, specialized models, and gateways. Provider-neutral execution policy becomes more valuable as the provider landscape becomes heterogeneous.

Second, context is larger but not free. Large context windows reduce some pressure but increase hidden waste. Prompt caching makes context layout itself an execution optimization problem [5, 6].

Third, agents are becoming long-running. Durable execution, state persistence, and human-in-the-loop workflows are increasingly part of agent infrastructure [4, 8, 15].

Fourth, tool boundaries are becoming protocolized. MCP turns tools into structured, discoverable interfaces rather than one-off function blobs [16].

Fifth, observability and evaluation are maturing. Traces, cost metrics, latency metrics, prompt management, datasets, and evaluator outputs are becoming production expectations [17, 18].

Together, these trends create the preconditions for an optimizer. Once execution becomes structured, measured, and multi-provider, optimization becomes possible.

16 Long-Term Vision

MIGAKI should not become another agent framework. Its long-term ambition is to establish execution optimization as an independent systems layer.

In mature AI systems:

Applications describe intent. Agent frameworks determine strategy. Execution optimizers determine realization. Workflow engines preserve durable state. Gateways provide provider connectivity. Observability systems record behavior. Evaluation systems measure outcomes. Models perform computation.
--

MIGAKI focuses on one responsibility:

Given a logical agentic workflow and explicit constraints, produce the best measurable execution plan and evidence for why it is acceptable.

17 Conclusion

Modern AI has invested extraordinary effort into increasing model intelligence. It has also produced an expanding ecosystem of agent frameworks, gateways, tool protocols, workflow engines, evaluation platforms, and observability systems. This is not a sign that execution is primitive. It is a sign that execution has become important enough for every layer to reinvent part of it.

When execution policy is scattered across application code, provider APIs, gateways, orchestration runtimes, and evaluation dashboards, optimization remains local. MIGAKI proposes a different structure:

Represent execution explicitly. Optimize it under constraints. Lower it through capability-aware backends. Measure every transformation. Export evidence.

The question is not whether AI systems will acquire execution engines. They already are. The question is whether those execution engines will remain fragmented, vendor-specific, and opaque, or whether agentic computation will acquire a portable execution representation that can be optimized, audited, and improved over time.

A One-Sentence Positioning

MIGAKI is an execution optimizer for agentic systems: it represents model/tool/retrieval/context workflows as portable execution graphs, transforms them under explicit cost, latency, quality, and governance constraints, lowers them into provider-specific runtimes, and emits evidence traces for every optimization.

B Claims Migaki Can Defend

- MIGAKI separates AI execution policy from application logic.
- MIGAKI represents model/tool/retrieval/context workflows as optimizable execution graphs.
- MIGAKI applies optimization passes under explicit cost, latency, quality, and governance constraints.
- MIGAKI lowers portable plans into provider-specific APIs through capability-aware backends.
- MIGAKI emits evidence traces so optimizations can be audited, replayed, and benchmarked.

C Claims Migaki Should Avoid

Avoid	Prefer
Same answer, lower cost.	Equivalent task outcome under declared validators and allowed regression thresholds.
AI compiler.	Execution optimizer for probabilistic model/tool graphs.
Provider-neutral execution across all models.	Portable logical plans with provider-specific lowering.
Replaces agent frameworks.	Runs beneath, beside, or inside agent frameworks through adapters.
Minimum confidence.	Minimum validated quality threshold.
Optimization certificates.	Execution evidence bundles.

References

- [1] OpenAI. *OpenAI Agents SDK: Agents*. <https://openai.github.io/openai-agents-python/agents/>. Accessed 2026-06-28.
- [2] OpenAI. *OpenAI Agents SDK for JavaScript/TypeScript*. <https://github.com/openai/openai-agents-js>. Accessed 2026-06-28.
- [3] OpenAI. *Tracing - OpenAI Agents SDK*. <https://openai.github.io/openai-agents-python/tracing/>. Accessed 2026-06-28.
- [4] OpenAI. *Running agents - OpenAI Agents SDK*. https://openai.github.io/openai-agents-python/running_agents/. Accessed 2026-06-28.
- [5] OpenAI. *Prompt caching - OpenAI API*. <https://developers.openai.com/api/docs/guides/prompt-caching>. Accessed 2026-06-28.

- [6] Anthropic. *Prompt caching - Claude Platform Docs*. <https://docs.anthropic.com/en/docs/build-with-claude/prompt-caching>. Accessed 2026-06-28.
- [7] Anthropic. *Manage costs effectively - Claude Code Docs*. <https://docs.anthropic.com/en/docs/claude-code/costs>. Accessed 2026-06-28.
- [8] LangChain. *LangGraph overview*. <https://docs.langchain.com/oss/python/langgraph/overview>. Accessed 2026-06-28.
- [9] Google Cloud. *Agent Development Kit documentation*. <https://docs.cloud.google.com/gemini-enterprise-agent-platform/build/adk>. Accessed 2026-06-28.
- [10] Microsoft. *Microsoft Agent Framework documentation*. <https://learn.microsoft.com/en-us/agent-framework/>. Accessed 2026-06-28.
- [11] LiteLLM. *Routing strategies - LiteLLM docs*. <https://docs.litellm.ai/docs/routing>. Accessed 2026-06-28.
- [12] Cloudflare. *Cloudflare AI Gateway*. <https://developers.cloudflare.com/ai-gateway/>. Accessed 2026-06-28.
- [13] Vercel. *AI Gateway observability*. <https://vercel.com/docs/ai-gateway/observability-and-spend/observability>. Accessed 2026-06-28.
- [14] Vercel. *Model fallbacks now available in Vercel AI Gateway*. <https://vercel.com/changelog/model-fallbacks-now-available-in-vercel-ai-gateway>. Accessed 2026-06-28.
- [15] Temporal. *Of course you can build dynamic AI agents with Temporal*. <https://temporal.io/blog/of-course-you-can-build-dynamic-ai-agents-with-temporal>. Accessed 2026-06-28.
- [16] Model Context Protocol. *Tools - Model Context Protocol Specification*. <https://modelcontextprotocol.io/specification/2025-03-26/server/tools>. Accessed 2026-06-28.
- [17] OpenTelemetry. *Semantic Conventions for Generative AI*. <https://github.com/open-telemetry/semantic-conventions-genai>. Accessed 2026-06-28.
- [18] Langfuse. *Langfuse documentation*. <https://langfuse.com/docs>. Accessed 2026-06-28.
- [19] Ong, I., Almahairi, A., Wu, V., Chiang, W.-L., Wu, T., Gonzalez, J. E., Kadous, M. W., and Stoica, I. RouteLLM: Learning to Route LLMs with Preference Data. *arXiv preprint arXiv:2406.18665*, 2024. <https://arxiv.org/abs/2406.18665>.
- [20] Li, H., Zhang, Y., Guo, Z., Wang, C., Tang, S., Zhang, Q., Yang, C., Qi, B., Ye, P., Bai, L., Wang, Z., and Hu, S. LLMRouterBench: A Massive Benchmark and Unified Framework for LLM Routing. *arXiv preprint arXiv:2601.07206*, 2026. <https://arxiv.org/abs/2601.07206>.